

README for K⁺ production study with NuMI

General Information

This document describes the tools used for single-bin cross section analysis for muon-neutrino-induced K⁺ production with NuMI run1 FHC data/MC.

- Internal note: <https://microboone-docdb.fnal.gov/cgi-bin/sso/ShowDocument?docid=43140>
- The thesis of this study can be found: **ADD URL WHEN ECOPY IS READY**

Location of the codes:

- Reconstruction Algorithm: https://github.com/nattani723/uB_KReco
- Analyser to fill ntuple: <https://github.com/nattani723/CCKaonProduction/>
- Analysis tools: <https://github.com/nattani723/CCKaonAnalysis>

Authors:

- Natsumi Taniuchi (author of this document, ntaniuchi@hep.phy.cam.ac.uk)
- The codes are made upon tools developed by Chris Thorpe (chris.thorpe@manchester.ac.uk) for Lambda production study, which codes can be found:
 - Lambda production ntuple: <https://github.com/cthorne123/HyperonProduction>
 - And its analysis tools: <https://github.com/cthorne123/HyperonAnalysis>
 - Lambda production publication PRL: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.130.231802>
- The general idea of reconstruction algorithm was developed by Isobel Mawby's (i.mawby1@lancaster.ac.uk) study of shower refining for CC nue events at DUNE.
 - Link to PhD thesis: <https://inspirehep.net/literature/2693856>

Running Reconstruction

Tutorial for the reconstruction can be found here:

https://microboone-docdb.fnal.gov/cgi-bin/sso/RetrieveFile?docid=41928&filename=KReco_rarexsec.pdf&version=1

Related codes can be found here: https://github.com/nattani723/uB_KReco

Setting up and Building

- Environment: `uboonecode v08_00_00_51 -q e17:prof`
- Get Packages: uboonecode, ubana
- Copy the code above to a subdirectory under ubana/ubana/

Input file: OverlayStageReco2 .root files

See the official list of fcl files for NuMI run1 FHC here:

https://cdcv.sfnal.gov/redmine/projects/uboone-physics-analysis/wiki/NuMI_Production_fcls

How to run the reconstruction

```
$lar -c CCKaonAnalyzerRebuild_mc.fcl -s input.root -o  
output.root
```

where the code of [CCKaonAnalyzerRebuild_mc.fcl](#) and its module can be found [here](#).

The module above can be broken down into three procedures.

1. Run module [CCKaonProducer_module.cc](#) by [CCKaonProducer_mc.fcl](#)

```
$lar -c CCKaonProducer_mc.fcl -s input.root -o output.root
```

 - This module runs the reconstruction algorithm and store the list of rebuilt `recob::Track` and associations with `recob::Hit`
2. Re-run the calorimetry for the newly built objects to obtain `anab::Calorimetry Objects`. This is done by re-running the necessary algorithm in `reco_uboone_data_mcc9_8.fcl`
3. Run module [CCKaonAnalyzer_module.cc](#) by [CCKaonAnalyzer_mc.fcl](#)

```
$lar -c CCKaonAnalyzer_mc.fcl -s input.root -o output.root
```

 - This makes a new track list including the rebuilt track by the algorithm (process 1) and obtain variables for BDT training such as track length, PID scores, and LLRPID scores and store as .root output
 - Run the analyser module introduced in the following section over the output rootfile

Producing Ntuple Files

Related codes can be found here: <https://github.com/nattani723/CCKaonProduction/>

Setting up

```
$source  
/cvmfs/uboone.opensciencegrid.org/products/setup_uboone_mcc9.sh  
$setup uboonecode v08_00_00_51 -q e17:prof
```

How to Run

The fhc files to run the analyser module are located under the **job** directory.

```
$lar -c run_KaonNtuplesSimple.fcl -s input.root -o output.root
```

General Idea

- The main module is **Modules/KaonNtuplesSimple_module.cc** which calls three submodules located under **Modules/Submodules/**
- The fhicl file to run the module is **job/run_KaonNtuplesSimple.fcl**. This calls the main module by

```
analyzers: { KaonNtuplesSimple: @local::analyseEvents }
```

with parameters being setup in the header file **kaonsimpleConfig.fcl**.
- To filter the events, run **job/run_KaonFilter.fcl**. This runs the filter module **Modules/KaonFilter_module.cc**, followed by filling the events into a ntuple file handled again by the main analyser module **Modules/KaonNtuplesSimple_module.cc**.

Analysing and Producing Plots

Related codes can be found here: <https://github.com/nattani723/CCKaonAnalysis>

Setting up and Building

Mostly cited from “README” of [lambda analysis code](#)

The code consists of two main folders:

- **Core/** containing objects that handle importing and manipulating events, recording metadata and management of the other algs and,
- **Alg/** storing the individual selection algorithms.

To get set up and build the code:

```
$source setup.sh
$make
```

General Description

This code uses the ntuples created by the **KaonNtuplesSimple_module.cc** LArSoft module and is designed to be run with root macros.

To import into a root macro, include the following to the top of your macro:

```
R__LOAD_LIBRARY($HYP_TOP/lib/libHyperon.so)
R__LOAD_LIBRARY($HYP_TOP/lib/libParticleDict.so)

#include "SelectionManager.h"
#include "EventAssembler.h"
#include "Cut.h"
#include "SelectionParameters.h"

#include "SampleSets_Example.h"
#include "Parameters.h"
```

This imports the shared objects, includes the corresponding headers and sets up the pre-defined parameter sets and sample sets.

A list of pre-sets for the selection are located in **ParameterSets/Parameters.h**. You can then set up the selection by doing SelectionManager M(P) where P is the parameter set you want to use.

Similarly, **SampleSets/SampleSets_Example.h** contains lists of ntuples typically used together (ie kaon+background+dirt+EXT+data).

Several examples of using this code in root macros exist in **examples/**

Useful Root Macros

- Run **AllSelectionVariablePlotList_AllSys.C** by

```
./run_AllSelectionVariablePlotList.sh
```

 - This plots data over MC after all selection with both statistical and systematic errors with as a distribution of an arbitrary variable stored in ntuple
- Run **SideBandVariablePlotList.C** by

```
./run_SidebandVariablePlotList.sh
```

 - This plots data over MC in sidebands with both statistical and systematic errors with as a distribution of an arbitrary variable stored in ntuple
- Files under **examples/General/**

- `SelectionPerformance.C`: check selection performance
- `VariableDistributionPlots.C`: plot distribution of variables stored in ntuples
- `BDTXCheckPlotList.C`: plot distribution of BDT variables. Run by `./run_BDTXCheckPlotList.sh`
- `PhaseSpaceTest.C`: check phase space dependent efficiency
- `MomentumPhaseSpaceCalculation.C`: check phase space threshold of momentum
- TMVA related, under `examples/TMVA/`
 - `FillTree.C`: this fills trees for BDT training after preselection
 - `TrainBDT.C`: this trains the BDT and makes an output of weight.
 - `BDTCutTest.C`: calculate the eff*purity, do not forget to update the weight input from trained TMVA.

To Calculate the Final Cross-section

1. Run `CrossSectionExtraction/SelectedSignalBG_ForSensitivity.C`
 - This calculates all the systematics except for detector variations (i.e. flux, generator, and reinteraction) and stores the variations in a root file.
2. Run `CrossSectionExtraction/Detector_Uncertainties_ForSensitivity.C`
 - This calculates the detector variations and stores the variations in a root file.
3. Run `CrossSectionExtraction/BayesianCrossSection.C`
 - Declare the output of the first process as:
`TFile *f`
`TFile *f_Systematics`
 - Declare the output of the second process as:
`TFile *f_Detector`

General Remarks

- Trained TMVA setup for NuMLHC Run1 only:
`TMVA::Types::kBDT, "BDT",
"!H:!V:NTrees=500:MinNodeSize=5%:MaxDepth=3:BoostType=AdaBoost:
AdaBoostBeta=0.60:UseBaggedBoost:BaggedSampleFraction=0.10:Sepa
rationType=GiniIndex:nCuts=10:DoBoostMonitor=true:UseRandomised
Trees=True:UseNvars=12:Shrinkage=0.001:PruneMethod=CostComplexi
ty:PruneStrength=3"`

MC Samples

General Remarks

Due to the limited statistics of ν -induced K^+ production events in the generic numi MC, the resampling production method, developed by Chris Thorpe, was used in this analysis.

The details of the procedure can be found here:

<https://microboone-docdb.fnal.gov/cgi-bin/sso/RetrieveFile?docid=42307&filename=Resampling%20Production%20Code-2.pdf&version=3>

The Strangeness samples, including K^+ , were produced by the production team. The campaign can be tracked here:

<https://docs.google.com/spreadsheets/d/13a4lgf99cVNudfZXObpqNGSIDnUxlnsy07c9x2arJWY/edit?gid=736358831#gid=736358831>

Note that all the plots shown in the thesis and the internal note are all produced with FHC Run1 samples only.

List of reco2 sample definitions after running the reconstruction algorithm

- CV:
taniuchi_genie_numi_nu_strange_sample_overlay_fhc_run1_CV_reco2_refined_KrecoAlg_parameter10_ReRunReco_reco
- LY Rayleigh:
taniuchi_genie_gen_events_numi_run1_fhc_detvar_LYRayleigh_overlay_resample_reco2_poststage_ReRunReco_reco
- LY Down:
taniuchi_genie_gen_events_numi_run1_fhc_detvar_LYDown_overlay_resample_reco2_poststage_ReRunReco_reco
- Wiremod X:
taniuchi_genie_gen_events_numi_run1_fhc_detvar_wiremodX_overlay_resample_reco2_poststage_ReRunReco_reco
- Wiremod YZ:
taniuchi_genie_gen_events_numi_run1_fhc_detvar_wiremodYZ_overlay_resample_reco2_poststage_ReRunReco_reco
- Wiremod ThetaXZ:
taniuchi_genie_gen_events_numi_run1_fhc_detvar_wiremodThetaXZ_overlay_resample_reco2_poststage_ReRunReco_reco
- Wiremod ThetaYZ:
taniuchi_genie_gen_events_numi_run1_fhc_detvar_wiremodThetaYZ_overlay_resample_reco2_poststage_ReRunReco_reco

- SCE:
taniuchi_genie_gen_events_num1_run1_fhc_detvar_SCE_overlay_resample_reco2_poststage_ReRunReco_reco
- Recomb2:
taniuchi_genie_gen_events_num1_run1_fhc_detvar_recomb_overlay_resample_reco2_poststage_ReRunReco_reco

List of reco2 sample definitions for BDT development with reconstruction algorithm and without cosmic (NOT overlaid)

- Ntuple files of Associated K+ with clear signals:
/exp/uboone/data/users/taniuchi/taniuchi/KaonNTuple/BDT/assock_events_num1_run1_fhc_signal_nocosmic_resample_reco2_ReRunReco_ntuple_all.root

I am not yet 100% sure the samdef of original reco2, which can be

- taniuchi_assock_events_num1_run1_fhc_signal_nocosmic_resample_reco2_ReRunReco_reco
- taniuchi_assock_events_num1_run1_fhc_signal_nocosmic_resample_reco2_ReRunReco_reco_pt2
- taniuchi_genie_gen_assock_events_num1_run1_fhc_signal_nocosmic_resample_reco2_ReRunReco_reco_pt3
- taniuchi_genie_gen_assock_events_num1_run1_fhc_single_nocosmic_resample_reco2_reco2_pt4
- Associated K+ that can be BG (ex, K+ reinteracted, inelastic proton, out FV etc):
 - /exp/uboone/data/users/taniuchi/taniuchi/KaonNTuple/BDT/assock_events_num1_run1_fhc_bg_nocosmic_resample_reco2_ReRunReco_ntuple_all.root
 - /exp/uboone/data/users/taniuchi/taniuchi/KaonNTuple/BDT/assock_events_num1_run1_fhc_bg_ProtonScatter_nocosmic_resample_reco2_ReRunReco_ntuple.root
 - /exp/uboone/data/users/taniuchi/taniuchi/KaonNTuple/BDT/assock_events_num1_run1_fhc_bg_ProtonInelastic_nocosmic_resample_reco2_ReRunReco_ntuple.root

I am not yet 100% sure the samdef of original reco2, which can be:

- taniuchi_genie_gen_assock_events_num1_run1_fhc_bg_nocosmic_resample_reco2_ReRunReco_reco
- taniuchi_genie_gen_assock_events_num1_run1_fhc_bg_nocosmic_resample_reco2_ReRunReco_reco_pt2
- taniuchi_genie_gen_assock_events_num1_run1_fhc_bg_nocosmic_resample_reco2_ReRunReco_reco_pt3
- taniuchi_genie_gen_assock_events_num1_run1_fhc_bg_nocosmic_resample_reco2_ReRunReco_reco_pt4
- taniuchi_genie_gen_assock_events_num1_run1_fhc_bg_nocosmic_resample_reco2_ReRunReco_reco_pt5

- taniuchi_genie_gen_assocK_events_num1_run1_fhc_bg_nocosmic_resample_reco2_reco2_pt6
- Single K+ with clear signals:
/exp/uboone/data/users/taniuchi/taniuchi/KaonNTuple/BDT/singleK_events_num1_run1_fhc_signal_nocosmic_reco2_ReRunReco_ntuple.root

I am not yet 100% sure the samdef of original reco2, which can be:

taniuchi_genie_gen_singleK_events_num1_run1_fhc_signal_nocosmic_resample_reco2_ReRunReco_reco

- Single K+ that can be BG:
/exp/uboone/data/users/taniuchi/taniuchi/KaonNTuple/BDT/singleK_events_num1_run1_fhc_bg_nocosmic_reco2_ReRunReco_ntuple.root

I am not yet 100% sure the samdef of original reco2, which can be:

taniuchi_singleK_events_num1_run1_fhc_bg_nocosmic_resample_reco2_ReRunReco_reco

- Generic NuMI:
/exp/uboone/data/users/taniuchi/taniuchi/KaonNTuple/BDT/numi_fhc_run1_sample012_refined_KrecoAlg_parameter10_ReRunReco_reco_ntuple.root

And its samdef:

- taniuchi_num1_sample0_refined_KrecoAlg_parameter10_ReRunReco_reco
- taniuchi_num1_sample1_refined_KrecoAlg_parameter10_ReRunReco_reco
- taniuchi_num1_sample2_refined_KrecoAlg_parameter10_ReRunReco_reco